# COMPLETE SQL NOTES

From Basics to Advanced

A practical guide to relational databases, SQL syntax, querying, joins, data modification, schema design, transactions, indexing, analytics, security, and performance.

Includes examples, practice questions, a mini-project, and quick reference.

Dialect note: SQL features differ among PostgreSQL, MySQL, SQL Server, Oracle, and SQLite. Examples use broadly familiar syntax; check your chosen database's documentation for exact compatibility.

# How to Use These Notes

Work through the chapters in order and run the examples in a practice database. SQL is declarative: you describe the result you want, and the database chooses a plan to produce it.

- Beginner: SELECT, filtering, sorting, expressions, NULL, aggregation, and joins.
- Intermediate: subqueries, DML/DDL, constraints, normalization, views, and transactions.
- Advanced: indexes, execution plans, CTEs, window functions, analytics, security, and optimization.

# Table of Contents


1. SQL and relational fundamentals
2. Data types and NULL
3. SELECT and logical query order
4. Filtering, sorting, and limiting
5. Expressions and built-in functions
6. Aggregation and GROUP BY
7. Joins
8. Subqueries and set operations
9. INSERT, UPDATE, DELETE, and upsert
10. DDL and schema objects
11. Keys and constraints
12. Database design and normalization
13. Views
14. Transactions and ACID
15. Indexes and query performance
16. CTEs and recursive queries
17. Window functions
18. Procedures, functions, and triggers
19. Security and SQL injection
20. Advanced analytics patterns
21. Dates, time zones, and JSON
22. Administration essentials
23. SQL dialect differences
24. Sales database mini-project
25. Practice questions and solutions
26. Quick reference and roadmap

# 1. SQL and Relational Database Fundamentals

## What is SQL?

SQL (Structured Query Language) is used to define, query, modify, and control data in relational database management systems (RDBMS). A table contains rows and columns; each row is a record and each column an attribute.

## Core vocabulary

- Database: organized collection of data and objects.
- Table: rows and columns representing a relation.
- Schema: logical namespace or structure of database objects.
- Primary key: column(s) uniquely identifying a row.
- Foreign key: column(s) referencing a key in another table.
- Query: SQL statement that reads or changes information.

## Common SQL categories

- DDL — CREATE, ALTER, DROP, TRUNCATE.
- DML — INSERT, UPDATE, DELETE; MERGE in supported systems.
- DQL — SELECT, a common label for querying data.
- DCL — GRANT and REVOKE.
- TCL — COMMIT, ROLLBACK, SAVEPOINT.

## Example schema

```
CREATE TABLE departments (
  department_id INTEGER PRIMARY KEY,
  department_name VARCHAR(100) NOT NULL
);
CREATE TABLE employees (
  employee_id INTEGER PRIMARY KEY,
  full_name VARCHAR(100) NOT NULL,
  department_id INTEGER,
  salary DECIMAL(10,2),
  hire_date DATE,
  manager_id INTEGER,
  FOREIGN KEY (department_id)
    REFERENCES departments(department_id)
);
```

SQL keywords are commonly capitalized for readability. Keywords are generally case-insensitive; identifier rules depend on the database.

# 2. Data Types and NULL

## Common types

- INTEGER / BIGINT: whole numbers.
- DECIMAL(p,s) / NUMERIC(p,s): exact decimal values, useful for money.
- REAL / FLOAT / DOUBLE: approximate floating-point values.
- CHAR(n), VARCHAR(n), TEXT: character data; availability varies.
- DATE, TIME, TIMESTAMP: temporal values.
- BOOLEAN: true/false where supported natively.
- BLOB / binary types: binary data.
- JSON / JSONB: JSON values in supporting databases.

### NULL handling

NULL means missing, unknown, or not applicable. It is not zero or an empty string. SQL uses three-valued logic: TRUE, FALSE, and UNKNOWN. WHERE keeps rows only when the condition is TRUE.

```
SELECT employee_id, full_name
FROM employees
WHERE department_id IS NULL;

SELECT full_name, COALESCE(salary, 0) AS salary_display
FROM employees;
```

Use IS NULL or IS NOT NULL, not = NULL. COALESCE returns the first non-NULL argument. NULLIF(a,b) returns NULL when a equals b and can prevent division by zero.

## 3. SELECT and Logical Query Order

```
SELECT full_name AS employee_name,
       salary * 12 AS estimated_annual_salary
FROM employees;
```

SELECT chooses output expressions; FROM identifies the source. Prefer explicit column lists over SELECT * in production queries when stable output and lower data transfer matter.

```
SELECT DISTINCT department_id
FROM employees;

SELECT full_name, salary
FROM employees
ORDER BY salary DESC, full_name ASC;
```

DISTINCT removes duplicate result rows. A useful conceptual processing order is FROM/JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT/OFFSET. This explains why a SELECT alias often cannot be used in WHERE.

### Comments

```
-- Single-line comment
/* Multi-line comment */
```

## 4. Filtering, Sorting, and Limiting

```
SELECT full_name, salary
FROM employees
WHERE salary >= 50000
  AND department_id IN (2, 3)
  AND full_name <> 'Sample Name';
```

Common comparison operators are =, <>, <, <=, >, >=. Combine conditions with AND, OR, and NOT; use parentheses when mixing AND and OR.

```
SELECT * FROM employees
WHERE salary BETWEEN 40000 AND 70000;

SELECT * FROM employees
WHERE full_name LIKE 'A%';
```

- BETWEEN includes both endpoints.
- IN tests membership in a list; NOT IN can behave unexpectedly if NULL appears in its subquery.
- LIKE uses % for any sequence and _ for one character in common dialects.
- ORDER BY supports ASC and DESC; add a deterministic tie-breaker for pagination.

```
-- Common in PostgreSQL, MySQL, SQLite
SELECT * FROM employees
ORDER BY employee_id
LIMIT 10 OFFSET 20;
```

SQL Server commonly uses TOP or OFFSET/FETCH. Other systems have their own row-limiting syntax.

# 5. Expressions and Built-in Functions

## Strings and numbers

```
SELECT UPPER(full_name), LOWER(full_name), LENGTH(full_name)
FROM employees;

SELECT ROUND(salary, 0), ABS(salary), FLOOR(salary)
FROM employees;
```

Common string functions include TRIM, SUBSTRING/SUBSTR, CONCAT, REPLACE, and length functions. Function names and argument conventions vary.

## CASE and conversion

```
SELECT full_name,
  CASE
    WHEN salary >= 90000 THEN 'High'
    WHEN salary >= 60000 THEN 'Medium'
    WHEN salary IS NULL THEN 'Unknown'
    ELSE 'Entry'
  END AS salary_band
FROM employees;

SELECT CAST('123' AS INTEGER);
```

CASE is a conditional expression. Use explicit casts when data types are ambiguous. Invalid conversion behavior differs by database.

## Dates

```
SELECT CURRENT_DATE, CURRENT_TIMESTAMP;
```

Date arithmetic, intervals, extraction functions, and time-zone handling vary by dialect. Choose types and conversion rules intentionally.

# 6. Aggregation and GROUP BY

## Aggregate functions

- COUNT(*) counts rows; COUNT(column) counts non-NULL values.
- SUM and AVG generally ignore NULL inputs.
- MIN and MAX return the minimum and maximum non-NULL values.

```
SELECT department_id,
       COUNT(*) AS employee_count,
       AVG(salary) AS average_salary,
       MAX(salary) AS highest_salary
FROM employees
GROUP BY department_id;
```

## WHERE versus HAVING

WHERE filters input rows before grouping. HAVING filters groups after aggregation.

```
SELECT department_id, AVG(salary) AS avg_salary
FROM employees
WHERE hire_date >= DATE '2022-01-01'
GROUP BY department_id
HAVING AVG(salary) > 60000;
```

## Conditional aggregation

```
SELECT department_id,
       COUNT(*) AS total,
       SUM(CASE WHEN salary >= 70000 THEN 1 ELSE 0 END) AS high_earners
FROM employees
GROUP BY department_id;
```

# 7. Joins

Joins combine rows according to a condition. Use table aliases and qualify overlapping column names.

## INNER and LEFT JOIN

```
SELECT e.full_name, d.department_name
FROM employees e
INNER JOIN departments d
  ON e.department_id = d.department_id;

SELECT d.department_name, e.full_name
FROM departments d
LEFT JOIN employees e
  ON e.department_id = d.department_id;
```

INNER JOIN returns matches only. LEFT JOIN keeps every row from the left table and fills unmatched right-side columns with NULL.

## Other joins and self joins

```
-- Every possible pair
SELECT d.department_name, e.full_name
FROM departments d CROSS JOIN employees e;

-- Employee and manager
SELECT e.full_name AS employee, m.full_name AS manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.employee_id;
```

RIGHT JOIN preserves all rows from the right side. FULL OUTER JOIN preserves unmatched rows from both sides but is not supported natively by every database.

## Common mistakes

- A missing or incomplete join condition can create a Cartesian product.
- Filtering a right-side column in WHERE after a LEFT JOIN may remove unmatched rows; place conditions in ON if unmatched left rows must remain.
- One-to-many joins can multiply rows; check the intended level of detail before aggregating.
- Prefer stable key columns over display names for joins.

# 8. Subqueries and Set Operations

```
SELECT full_name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);

SELECT e.full_name
FROM employees e
WHERE EXISTS (
  SELECT 1 FROM projects p
  WHERE p.owner_employee_id = e.employee_id
);
```

A scalar subquery returns one value. EXISTS tests whether a matching row exists. Correlated subqueries refer to the outer row; optimizers may transform them into joins or other plans.

```
SELECT d.department_id
FROM departments d
WHERE NOT EXISTS (
  SELECT 1 FROM employees e
  WHERE e.department_id = d.department_id
);
```

NOT EXISTS is often safer than NOT IN when the subquery may return NULL.

```
SELECT city FROM customers
UNION
SELECT city FROM suppliers;
```

```
SELECT city FROM customers
UNION ALL
SELECT city FROM suppliers;
```

UNION removes duplicates; UNION ALL preserves them and is often faster. INTERSECT returns common rows; EXCEPT returns rows from the first query absent from the second. Some systems use MINUS. Branches need compatible column counts and types.

# 9. INSERT, UPDATE, DELETE, and Upsert

```
INSERT INTO departments (department_id, department_name)
VALUES (10, 'Research');

INSERT INTO employees (employee_id, full_name, department_id, salary)
VALUES (101, 'Asha Rao', 10, 65000),
       (102, 'Ravi Kumar', 10, 72000);

UPDATE employees
SET salary = salary * 1.05
WHERE department_id = 10;

DELETE FROM employees
WHERE employee_id = 102;
```

Without a WHERE clause, UPDATE changes every row and DELETE removes every row. Preview the target with SELECT first. Use transactions and backups for important modifications.

## DELETE versus TRUNCATE

DELETE removes selected rows and may fire row-level triggers. TRUNCATE removes all rows efficiently in many systems, but transaction, trigger, identity-reset, and permission behavior differ.

## Upsert

```
-- PostgreSQL-style example
INSERT INTO departments (department_id, department_name)
VALUES (10, 'Research')
ON CONFLICT (department_id)
DO UPDATE SET department_name = EXCLUDED.department_name;
```

Upsert syntax differs: PostgreSQL/SQLite use ON CONFLICT, MySQL uses ON DUPLICATE KEY UPDATE, and SQL Server/Oracle support MERGE with product-specific semantics.

# 10. DDL and Schema Objects

```
CREATE TABLE projects (
  project_id INTEGER PRIMARY KEY,
  project_name VARCHAR(120) NOT NULL,
  budget DECIMAL(12,2) DEFAULT 0
);

ALTER TABLE projects ADD COLUMN status VARCHAR(20);
DROP TABLE projects;
```

CREATE defines an object, ALTER changes its structure, and DROP removes the object. DDL transaction behavior varies. Temporary tables and generated identity/sequence syntax also differ.

## Naming conventions

- Use clear, consistent table and column names.
- Avoid reserved keywords as identifiers.
- Choose one naming style and use it consistently.
- Define numeric precision, text lengths, units, and defaults deliberately.

# 11. Keys, Constraints, and Data Integrity

```
CREATE TABLE customers (
  customer_id INTEGER PRIMARY KEY,
  email VARCHAR(255) UNIQUE NOT NULL
);

CREATE TABLE products (
  product_id INTEGER PRIMARY KEY,
  product_name VARCHAR(150) NOT NULL,
  sku VARCHAR(40) UNIQUE,
  price DECIMAL(10,2) NOT NULL CHECK (price >= 0)
);
```

Primary keys uniquely identify rows and cannot be NULL. UNIQUE prevents duplicate key values; NOT NULL requires a value; CHECK validates a condition; DEFAULT supplies an omitted value. CHECK constraints and NULL semantics deserve attention.

```
CREATE TABLE order_items (
  order_id INTEGER NOT NULL,
  product_id INTEGER NOT NULL,
  quantity INTEGER NOT NULL CHECK (quantity > 0),
  unit_price DECIMAL(10,2) NOT NULL CHECK (unit_price >= 0),
  PRIMARY KEY (order_id, product_id)
);
```

Composite keys contain multiple columns. Foreign keys preserve referential integrity. ON DELETE actions may include CASCADE, SET NULL, SET DEFAULT, RESTRICT, or NO ACTION depending on engine; use CASCADE deliberately.

# 12. Database Design and Normalization

Normalization reduces duplication and update anomalies. It is a design tool, not a rule that every query must use as many tables as possible.

- 1NF: values are atomic at the intended level; avoid repeating groups.
- 2NF: in 1NF, and every non-key attribute depends on the whole candidate key, not just part of a composite key.
- 3NF: in 2NF, with no inappropriate transitive dependency of non-key attributes on a candidate key.
- BCNF: every determinant of a non-trivial functional dependency is a candidate key.

## Relationships

- One-to-one: each record relates to at most one record on the other side.
- One-to-many: one department can have many employees.
- Many-to-many: use a junction table, e.g., student_courses(student_id, course_id).

Store department data in a departments table instead of repeating department names in every employee record. Denormalization intentionally duplicates or precomputes data for performance, but requires a consistency strategy.

# 13. Views and Materialized Views

```
CREATE VIEW high_paid_employees AS
SELECT employee_id, full_name, salary
FROM employees
WHERE salary >= 80000;

SELECT * FROM high_paid_employees;
```

A standard view stores a query definition, not necessarily its result. It helps reuse and abstraction but does not automatically make a query faster. A materialized view stores results and must be refreshed or maintained; support varies.

- Encapsulate common joins and calculations.
- Provide a stable interface for reporting.
- Expose only intended columns, alongside appropriate permissions.

- Do not assume views are a security boundary without checking execution context and database rules.

# 14. Transactions and ACID

```
BEGIN;
UPDATE accounts SET balance = balance - 100 WHERE account_id = 1;
UPDATE accounts SET balance = balance + 100 WHERE account_id = 2;
COMMIT;
-- Use ROLLBACK instead if validation fails.
```

A transaction groups changes into a logical unit. A real transfer also needs sufficient-funds checks and concurrency-safe logic.

- Atomicity: all-or-nothing operation.
- Consistency: constraints and invariants remain valid.
- Isolation: concurrent transactions are controlled to limit interference.
- Durability: committed changes survive failures according to system guarantees.

Common isolation level names include READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ, and SERIALIZABLE. Defaults and exact behavior vary. Anomalies include dirty reads, non-repeatable reads, phantom reads, and serialization conflicts.

```
BEGIN;
UPDATE employees SET salary = salary * 1.02;
SAVEPOINT after_raise;
UPDATE employees SET salary = -1 WHERE employee_id = 101;
ROLLBACK TO SAVEPOINT after_raise;
COMMIT;
```

Savepoint syntax varies. Keep transactions reasonably short to limit lock contention.

# 15. Indexes and Query Performance

An index is a data structure that can locate rows without scanning an entire table. It can accelerate reads but consumes storage and adds maintenance work to INSERT, UPDATE, and DELETE.

```
CREATE INDEX idx_employees_department
ON employees(department_id);

CREATE INDEX idx_employees_department_salary
ON employees(department_id, salary);

EXPLAIN
SELECT employee_id, full_name
FROM employees
WHERE department_id = 10;
```

## Index principles

- Index columns used often in selective filters, joins, and ordering when measurements justify it.
- Composite index column order matters; (department_id, salary) often helps queries beginning with department_id.
- Avoid many overlapping indexes without measuring the workload.
- Functions or casts on indexed columns may prevent normal index use unless a matching expression index is available.
- Indexes add write cost; test with real data and execution plans.

EXPLAIN (or the engine's equivalent) shows a query plan: scans, join algorithms, estimated rows, and possible index use. Actual plans may show timing and real row counts. Keep optimizer statistics current. A sargable predicate is one that can use an index search efficiently.

# 16. CTEs and Recursive Queries

```
WITH department_totals AS (
  SELECT department_id, COUNT(*) AS employee_count
  FROM employees GROUP BY department_id
)
SELECT d.department_name, t.employee_count
FROM departments d
JOIN department_totals t
  ON t.department_id = d.department_id;
```

A common table expression (CTE) names a subquery for one statement. It improves readability and supports stepwise transformations. Materialization or inlining behavior depends on the engine.

```
WITH RECURSIVE org AS (
  SELECT employee_id, full_name, manager_id, 0 AS depth
  FROM employees WHERE manager_id IS NULL
  UNION ALL
  SELECT e.employee_id, e.full_name, e.manager_id, o.depth + 1
  FROM employees e JOIN org o ON e.manager_id = o.employee_id
)
SELECT * FROM org;
```

Recursive CTEs traverse hierarchies and graphs. Prevent cycles or set a depth limit where cycles are possible. Syntax differs by system.

# 17. Window Functions

Window functions calculate across related rows while preserving each input row. GROUP BY usually collapses each group to one row.

```
SELECT full_name, department_id, salary,
  ROW_NUMBER() OVER (
    PARTITION BY department_id ORDER BY salary DESC
  ) AS row_num,
  RANK() OVER (
    PARTITION BY department_id ORDER BY salary DESC
  ) AS salary_rank
FROM employees;
```

ROW_NUMBER assigns unique sequential numbers. RANK leaves gaps after ties; DENSE_RANK does not. Add a tie-breaker for deterministic order.

```
SELECT order_date, amount,
  SUM(amount) OVER (
    ORDER BY order_date
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
  ) AS running_total,
  AVG(amount) OVER (
    ORDER BY order_date
    ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
  ) AS seven_row_average
FROM daily_sales;
```

```
SELECT customer_id, order_date, amount,
  LAG(amount) OVER (
    PARTITION BY customer_id ORDER BY order_date
  ) AS previous_amount,
  LEAD(amount) OVER (
    PARTITION BY customer_id ORDER BY order_date
  ) AS next_amount
FROM orders;
```

The seven-row average is based on rows, not necessarily seven calendar days. Specify window frames when required; default frames can surprise, particularly with duplicate ordering values.

# 18. Procedures, Functions, and Triggers

Stored routines execute logic in the database. They can centralize rules and reduce network round trips, but can complicate testing and migration if overused. Creation syntax, parameter modes, and return types differ substantially among engines.

Triggers run automatically on events such as INSERT, UPDATE, or DELETE. Uses include auditing and maintaining derived data. They can hide side effects, so document and test them.

```
CREATE TABLE employee_salary_audit (
  audit_id INTEGER PRIMARY KEY,
  employee_id INTEGER NOT NULL,
  old_salary DECIMAL(10,2),
  new_salary DECIMAL(10,2),
  changed_at TIMESTAMP
);
```

This is an audit-table design example, not a complete trigger. Trigger syntax and key/timestamp generation are dialect-specific.

# 19. Security and SQL Injection

Apply least privilege: grant users only the access they need. Separate application, reporting, deployment, and administrative roles when practical.

```
-- Example syntax; exact permissions vary
GRANT SELECT ON employees TO analyst_role;
REVOKE UPDATE ON employees FROM analyst_role;
```

## SQL injection prevention

SQL injection occurs when untrusted input is interpreted as SQL syntax. Use parameterized queries/prepared statements; never concatenate raw user input into SQL. Dynamic identifiers such as table names often cannot be bound as values, so validate them against a strict allowlist.

```
-- Application-code pseudocode
SELECT employee_id, full_name
FROM employees
WHERE department_id = ?;
```

- Use managed secrets and avoid passwords in source code.
- Encrypt connections and backups when appropriate.
- Patch the database and restrict network exposure.
- Audit privileged actions and review grants.
- Avoid returning unnecessary confidential fields.

# 20. Advanced Analytics Patterns

## Top N per group

```
WITH ranked AS (
  SELECT e.*,
    ROW_NUMBER() OVER (
      PARTITION BY department_id
      ORDER BY salary DESC, employee_id
    ) AS rn
  FROM employees e
)
SELECT * FROM ranked WHERE rn <= 3;
```

## Find duplicates

```
SELECT email, COUNT(*) AS occurrences
FROM customers
WHERE email IS NOT NULL
GROUP BY email
HAVING COUNT(*) > 1;
```

### Ratios and conditional pivot

```
SELECT department_id,
  COUNT(*) AS employee_count,
  100.0 * COUNT(*) /
    NULLIF((SELECT COUNT(*) FROM employees), 0) AS pct_of_all
FROM employees GROUP BY department_id;
```

Use a decimal literal when integer division would truncate. NULLIF prevents division by zero. Pivoting turns category values into columns; conditional aggregation with SUM(CASE WHEN category=... THEN value ELSE 0 END) is a portable approach.

### Gaps, islands, and pagination

Gaps-and-islands problems identify consecutive runs or missing ranges. Common approaches use ROW_NUMBER or LAG to mark boundaries. For large datasets, keyset pagination often scales better than large OFFSET values and requires stable unique ordering.

```
SELECT employee_id, full_name
FROM employees
WHERE employee_id > :last_seen_id
ORDER BY employee_id
FETCH FIRST 50 ROWS ONLY;
```

The placeholder and FETCH FIRST syntax are examples; use the target driver's parameter style and database syntax.

## 21. Dates, Time Zones, and JSON

```
SELECT *
FROM orders
WHERE order_timestamp >= :start_time
  AND order_timestamp < :end_time;
```

Half-open intervals [start, end) avoid double-counting boundary instants across adjacent periods. Distinguish local wall-clock time from an absolute instant; daylight-saving transitions can make local times ambiguous.

Databases differ in time-zone-aware timestamp semantics and conversion functions. PostgreSQL supports JSON/JSONB; MySQL, SQL Server, and SQLite have their own JSON capabilities and version requirements.

- Keep stable, frequently queried attributes in typed columns.
- Validate JSON shape in the application or database.
- Document schema evolution and missing-key versus NULL semantics.
- Consider generated/computed columns and indexes for frequently queried JSON paths.

## 22. Administration Essentials

### Backups and recovery

A backup is useful only if it can be restored. Define recovery point objective (RPO) and recovery time objective (RTO), test restores, and understand full, incremental, and log/WAL-based recovery for the chosen engine.

### Migrations and monitoring

- Version schema changes in source control.
- Test migrations on representative data and staging systems.
- Plan for locks, long updates, rollback, and application compatibility.
- Monitor latency, throughput, connections, lock waits, deadlocks, disk space, replication lag, and failed jobs.
- Replication can improve availability and read scale but is not a backup. Understand lag and failover behavior.

# 23. SQL Dialect Differences

| Feature | Variations to check |
|---|---|
| Row limiting | LIMIT/OFFSET; TOP; OFFSET/FETCH; FETCH FIRST |
| Upsert | ON CONFLICT; ON DUPLICATE KEY UPDATE; MERGE |
| Identity columns | IDENTITY; AUTO_INCREMENT; sequences/generated identity |
| String concatenation | CONCAT(); ||; + in some SQL Server contexts |
| Date functions | DATEADD/DATEDIFF; INTERVAL; DATE_PART/EXTRACT; vendor functions |
| Boolean | Native BOOLEAN or alternative numeric/bit types |
| JSON | Different types, operators, indexing, and validation |
| Stored routines | Different languages, delimiters, parameter modes, transaction rules |
| FULL OUTER JOIN | Native support varies |

Identify the target database and version before relying on advanced syntax. Isolate dialect-specific queries and test against the production engine.

# 24. Mini-Project: Sales Database

Goal: model customers, products, orders, and order items, then answer business questions. Adapt identity and timestamp types to your database.

```
CREATE TABLE customers (
  customer_id INTEGER PRIMARY KEY,
  customer_name VARCHAR(120) NOT NULL,
  email VARCHAR(255) UNIQUE,
  created_at TIMESTAMP
);
CREATE TABLE products (
  product_id INTEGER PRIMARY KEY,
  product_name VARCHAR(120) NOT NULL,
  unit_price DECIMAL(10,2) NOT NULL CHECK (unit_price >= 0),
  active_flag INTEGER NOT NULL DEFAULT 1
);
CREATE TABLE orders (
  order_id INTEGER PRIMARY KEY,
  customer_id INTEGER NOT NULL,
  order_date DATE NOT NULL,
  status VARCHAR(20) NOT NULL,
  FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);
CREATE TABLE order_items (
  order_id INTEGER NOT NULL,
  product_id INTEGER NOT NULL,
  quantity INTEGER NOT NULL CHECK (quantity > 0),
  unit_price DECIMAL(10,2) NOT NULL CHECK (unit_price >= 0),
  PRIMARY KEY (order_id, product_id),
  FOREIGN KEY (order_id) REFERENCES orders(order_id),
  FOREIGN KEY (product_id) REFERENCES products(product_id)
);
```

## Order totals

```
SELECT o.order_id, o.order_date,
       SUM(oi.quantity * oi.unit_price) AS order_total
FROM orders o JOIN order_items oi ON oi.order_id = o.order_id
GROUP BY o.order_id, o.order_date
ORDER BY o.order_date DESC;
```

### Customer lifetime spend

```
SELECT c.customer_id, c.customer_name,
       COALESCE(SUM(oi.quantity * oi.unit_price), 0) AS lifetime_spend
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
LEFT JOIN order_items oi ON oi.order_id = o.order_id
GROUP BY c.customer_id, c.customer_name
ORDER BY lifetime_spend DESC;
```

### Best-selling products

```
SELECT p.product_id, p.product_name, SUM(oi.quantity) AS units_sold
FROM products p
JOIN order_items oi ON oi.product_id = p.product_id
JOIN orders o ON o.order_id = oi.order_id
WHERE o.status = 'completed'
GROUP BY p.product_id, p.product_name
ORDER BY units_sold DESC
FETCH FIRST 10 ROWS ONLY;
```

### Monthly revenue

```
SELECT EXTRACT(YEAR FROM o.order_date) AS sales_year,
       EXTRACT(MONTH FROM o.order_date) AS sales_month,
       SUM(oi.quantity * oi.unit_price) AS revenue
FROM orders o JOIN order_items oi ON oi.order_id = o.order_id
WHERE o.status = 'completed'
GROUP BY EXTRACT(YEAR FROM o.order_date),
         EXTRACT(MONTH FROM o.order_date)
ORDER BY sales_year, sales_month;
```

A real system must define treatment of cancellations, refunds, discounts, tax, shipping, and currency. Add sample data, indexes, views, status constraints, and tests as extensions.

# 25. Practice Questions and Solutions

- 1. Return employees earning over 60,000.
- 2. Count employees in each department.
- 3. Show all departments, including those with zero employees.
- 4. Find employees earning above the company average.
- 5. Find departments whose average salary exceeds 70,000.
- 6. Return the three highest-paid employees per department.
- 7. Find customers who never placed an order.
- 8. Find duplicate emails.
- 9. Calculate a running total of sales by date.
- 10. Why is salary = NULL incorrect?
- 11. Explain WHERE versus HAVING.
- 12. Describe the trade-off of adding an index.
- 13. Outline a safe bulk UPDATE workflow.
- 14. How do parameterized queries reduce injection risk?
- 15. Name the ACID properties.

### Solutions

```
-- 1
SELECT * FROM employees WHERE salary > 60000;
-- 2
SELECT department_id, COUNT(*) FROM employees GROUP BY department_id;
-- 3
SELECT d.department_id, d.department_name, COUNT(e.employee_id)
FROM departments d LEFT JOIN employees e
ON e.department_id = d.department_id
GROUP BY d.department_id, d.department_name;
-- 4
SELECT * FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
-- 5
SELECT department_id, AVG(salary) FROM employees
GROUP BY department_id HAVING AVG(salary) > 70000;
-- 6
WITH ranked AS (
 SELECT e.*, ROW_NUMBER() OVER (
   PARTITION BY department_id ORDER BY salary DESC, employee_id
 ) AS rn FROM employees e
)
SELECT * FROM ranked WHERE rn <= 3;
-- 7
SELECT c.* FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id=c.customer_id);
-- 8
SELECT email, COUNT(*) FROM customers
WHERE email IS NOT NULL GROUP BY email HAVING COUNT(*) > 1;
-- 9
SELECT sale_date, amount, SUM(amount) OVER (
 ORDER BY sale_date ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
) AS running_total FROM sales;
-- 10
-- Use WHERE salary IS NULL.
```

11. WHERE filters rows before grouping; HAVING filters groups. 12. Indexes may speed reads but consume storage and add write overhead. 13. Preview rows with SELECT, back up important data, use a transaction when appropriate, test in staging, verify row counts, and validate before committing. 14. Bound parameters separate values from SQL syntax. 15. Atomicity, Consistency, Isolation, Durability.

# 26. Quick Reference and Learning Roadmap

| Task | Pattern |
|---|---|
| Read | SELECT columns FROM table WHERE condition; |
| Sort | ORDER BY column ASC/DESC |
| Aggregate | GROUP BY columns HAVING aggregate_condition |
| Join | FROM a JOIN b ON a.key = b.key |
| Insert | INSERT INTO table (columns) VALUES (values); |
| Update | UPDATE table SET column=value WHERE condition; |
| Delete | DELETE FROM table WHERE condition; |
| Create | CREATE TABLE table_name (...); |
| Index | CREATE INDEX index_name ON table_name(column); |
| Transaction | BEGIN; ... COMMIT; or ROLLBACK; |
| CTE | WITH name AS (SELECT ...) SELECT ...; |
| Window | function() OVER (PARTITION BY ... ORDER BY ...) |
| NULL test | column IS NULL / IS NOT NULL |

## Suggested six-week plan

- Week 1: SELECT, WHERE, ORDER BY, expressions, NULL.
- Week 2: aggregates, GROUP BY, HAVING, joins, set operations.
- Week 3: subqueries, DML/DDL, constraints, normalization.
- Week 4: views, transactions, indexes, EXPLAIN, CTEs.
- Week 5: window functions, recursive queries, analytics.
- Week 6: permissions, parameterization, migrations, backups, and a project.

## Final checklist

- Predict query results before running them.
- Choose joins correctly and understand their effect on row counts.
- Handle NULL correctly.
- Write grouped and window-function queries.
- Use constraints to protect data quality.
- Read execution plans and measure index effects.
- Modify data safely and use transactions correctly.
- Adapt syntax to the database engine and version.

---

End of notes. Practice against a real database, consult the documentation for your chosen SQL engine, and verify behavior with tests.